

Table of Contents

Table of Contents	1
<i>Abstract</i> —	4
2. Introduction and Nepali Context.....	4
2.1 The Undigitised Record Crisis.....	4
2.2 Why Manual Transcription Cannot Scale.....	4
2.3 Why Computer Vision and NLP Are the Appropriate Approach.....	5
2.4 Scope.....	5
3. Critical Analysis of Existing Solutions.....	5
3.1 Tesseract OCR v4 — Off-the-Shelf Printed Text Recogniser.....	5
Architecture and Training Data	5
Performance and Limitations	5
Contribution to System Design.....	6
3.2 VGG16 as a Standalone CNN Classifier — Strong Backbone, Wrong Head.....	6
Architecture and Training Data	6
Performance and Limitations	6
Contribution to System Design.....	6
3.3 EasyOCR (CRAFT + CRNN) — Architecture Confirmation and Deployment Lessons.....	6
Architecture and Training Data	6
Performance and Limitations	6
Contribution to System Design.....	6
3.4 Structured Comparison	7
3.5 Synthesis	8
4. Proposed Innovative Solution.....	8
4.1 System Overview — Seven-Stage Pipeline.....	8
4.2 Stages 1–2: Reference Setup and Input Preprocessing	8
4.3 Stage 3: Template Alignment via ORB and RANSAC	8
4.4 Stage 4: Region-of-Interest Cropping.....	9

4.5 Stage 5: CRNN Text Recognition with CTC Decoding	9
4.6 Two-Phase Transfer Learning Strategy	9
4.7 Stage 6: NLP Post-Processing	10
5. Implementation and Methodology	10
5.1 Datasets	10
Phase 1 — HindiSeg (Sabarinathan, n.d.).....	10
Phase 2 — Nepali Font-Generated Names (kritakhere, n.d.)	10
Train/Validation/Test Split	11
5.2 Preprocessing Pipeline	11
5.3 Data Augmentation	11
5.4 Model Architecture Summary.....	11
5.5 Training Configuration	12
6. Experimental Results and Evaluation	12
6.1 Evaluation Metrics and Justification.....	12
6.2 Full Pipeline Performance.....	12
6.3 NLP Post-Processor Ablation — Real Handwritten Forms.....	13
6.4 Error Analysis	13
Matra Confusion	13
Place of Birth Recognition.....	16
Conjunct Substitutions	16
6.5 Discussion	16
7. Honest Self-Critique	17
7.1 What the System Does Well	17
7.2 Genuine Limitations.....	17
Synthetic-Only Phase 2 Training	17
Accuracy Insufficient for Autonomous Deployment.....	17
Matra and Diacritic Recognition.....	17

Evaluation Sample Size	17
7.3 Concrete Improvement Proposals	17
8. Deployment and Practical Considerations	17
8.1 Hardware and Software Requirements	17
8.2 Connectivity Requirements.....	17
8.3 User Interface and Literacy Considerations.....	18
8.4 Cost Estimate	18
8.5 Data Privacy and Equity Considerations	18
10. Conclusion and Future Work.....	18
10.1 Summary	18
10.2 Future Work.....	18
11. References.....	20
Appendix A — Contribution Log	22
Appendix B — Meeting Minutes.....	23
Appendix D — AI Tool Usage Declaration	31

Handwritten OCR for Nepali Government Forms: A CRNN Pipeline with NLP Post-Processing

Kritak Aryal

Student ID: 0382309

Rashmi Gauli

Student ID: 0382376

Junisha Shakya

Student ID: 0382339

Noel Maharjan

Student ID: 0382303

*Bachelor of Computer Science(hons), IIMS College, Naxal, Kathmandu, Nepal
(in affiliation with Taylor's University, Malaysia)*

Abstract —

Handwritten government documents, such as those pertaining to citizenship, land registration, courts, etc., are extremely common in Nepal. Only a handful of them are digital in Nepal. Hand typing them is the obvious solution, but it can be costly and time-consuming, and it is not scalable. In order to address this, we have developed an end-to-end handwritten Devanagari OCR pipeline that uses computer vision and natural language processing (NLP) technologies to read and correct the Devanagari government form images from the Nepali government. In order to get around this, we suggest an end-to-end handwritten Devanagari OCR pipeline that uses computer vision and natural language processing technologies to automatically read and correct Devanagari text from fixed template Nepali government form images. Nine handwritten field regions are cropped out and fed into a Convolutional Recurrent Neural Network (CRNN) with CTC decoding after the scanned form has been correctly aligned with the reference template (Shi et al. (2016) Graves and associates). The system generates the raw text without the need to separate the image into its constituent characters. Subsequently, systematic errors are eliminated through a Unicode normalization process and fuzzy matching with Nepali vocabulary. The model can be pre-trained because Devanagari script is used to write both Hindi and Nepali. In the first phase, real handwritten Hindi images were used for training, and in the second phase, synthetic Nepali names with varying learning rates were used. In the field, the accuracy of words (from the Nepali validation set) was 91.87% after post-processing, while the accuracy of actual forms ranged from 78 to 89%. Fine-tuning raised Nepali test word accuracy from 47.15% to 69.92%, a gain of 22.77 points over the Hindi-only baseline, and post-processing raised it further to 91.87%. It will assist with the digitization of the first pass, but it won't be entirely automated; individuals will need to verify it.

2. Introduction and Nepali Context

2.1 The Undigitised Record

One of the oldest systems of handwritten government records in South Asia is found in Nepal, which has a long history of doing so. Every year, the state archives millions of records in more than 77 districts. These records include citizenship certificates (nagarikta praman patra), land ownership documents (lalpurja), civil service records, and court records, all of which are filled out by hand in Devanagari script. The highest authority for preserving documents 25 years or older is the National Archives of Nepal, which was founded under the Archives Conservation Act 1989.

In order to secure their rights to inherit land, obtain a passport, enroll their children in school, and receive subsidized healthcare, ordinary Nepalis, particularly those living in rural hill and mountain regions, require authentic

government documents. With over 17 million citizens registered in the National ID program by 2025, Nepal's civil registration system is transitioning from manually maintained paper documents to an electronic system (Department of National ID and Civil Registration [DONIDCR], 2023). However, coverage gaps are greatest in rural areas. In the absence of digitized underlying paper records, citizens must go through drawn-out re-verification procedures if they are lost or damaged (typically as a result of monsoon flooding, earthquakes, or improper storage). Particularly in rural areas, women are disadvantaged; according to the International Organization for Migration [IOM], 2016, only 19.7% of Nepali women have documentation proving they own land, compared to 51.5% of the country's female population.

2.2 Why Manual Transcription Cannot Scale

Government data entry employees manually enter the data in Nepal's public sector's current manual digitization

process. This method's primary scaling constraint is linear throughput, which means that as more records are generated, more will be backed up. Public service delivery in Nepal is still primarily paper-based despite more than 20 years of e-governance initiatives since the IT Policy 2000. The country's public service digitalization faces a number of obstacles, including low institutional capacity, inadequate infrastructure, and digital literacy (Rajkarnikar, 2025). This requirement is not met by the current general-purpose optical character recognition (OCR) systems. For example, Tesseract OCR works well on printed Devanagari fonts, but it is not appropriate for handwritten input because the ink spread and stroke variability differ from the printed training corpus that Tesseract uses (Smith, 2007).

2.3 Why Computer Vision and NLP Are the Appropriate Approach

The current DL architectures directly address every issue related to the visual sequence labeling problem of handwritten Devanagari text recognition. Devanagari's compositional structure consists of base consonants, vowel diacritical marks (matras), conjunct consonants, and modifier marks. This makes it challenging to recognize a single handwritten word and necessitates an understanding of both the visual pattern and the sequential dependence of the strokes. One method that creates hierarchical visual representations that are readily applicable to handwriting recognition is the Convolutional Neural Network (CNN). Bidirectional Long Short-Term Memory networks are crucial for Devanagari conjuncts, where a character's identity depends on its neighboring characters. They can learn both forwards and backwards (left-to-right and right-to-left). On the other hand, Connectionist Temporal Classification (CTC) loss learns the implicit mapping between image time-steps and character outputs without the need for character level segmentation (Graves et al. This is crucial for Devanagari because conjunct boundaries are difficult to identify using a rule-based method (2006). Lastly, vocabulary-guided fuzzy matching is used in NLP post-processing to fix systematic and predictable OCR errors without having to retrain the model.

2.4 Scope

The Devanagari OCR problem is addressed in this project in a constrained and repeatable manner. The system is designed for one single fixed template form (the Red Minimalist Membership Form A4), where the following nine fields are handwritten in Devanagari script: first name, last name, place of birth, father's first name, father's last name, mother's first name, mother's last name, nationality, district. One benefit of the fixed-template approach is that it can extract field coordinates without the need for a text detection model, provided that the shape is consistent in the template. Multiple-page documents, mixed-script documents, free form layouts, and variations of printed text cannot be processed by the system. These limitations will define a problem that is manageable, rigorously assessable,

and demonstrates the fundamental technical methodology without exaggerating its generalizability.

3. Critical Analysis of Existing Solutions

Before designing the proposed system, the team has done a well-established overview of 3 current AI answers that solve related troubles of optical character recognition and script reputation of Devanagari paragraph. The answers of three different models have been selected as they come from different deployment options, ranging from mature and ready-to-use models (3 answers: 3 Tesseract OCR v4, VGG16 as standalone CNN classifier) to deep learning architectures which may be very similar to the proposed answer (EasyOCR: CRAFT + CRNN). In this part structure of the system, overall performance, scientifically proven merits, challenges faced in handwritten Nepali shape processing and ideas borrowed from the various responses to the final system design are discussed for each answer.

3.1 Tesseract OCR v4 ; Off-the-Shelf Printed Text Recogniser

Architecture and Training Data

One of the most widely-used open source OCR engines around the world is tesseract OCR, which was originally developed at Hewlett-Packard and is maintained by Google (Smith, 2007). Inversion 4.0 improves the classic page-segmentation pipeline by adding a character recogniser based on a Long Short-Term Memory (LSTM) network. The engine's working phases are adaptive page layout analysis, sequential learning of characters by LSTM networks and a post processor using n-gram statistics for the language. There is a special Nepali language pack (nep) trained with printed Devanagari corpus data.

Performance and Limitations

Tesseract v4 can perform over 95% accurate character recognition on clean printed text (Smith, 2007). But on handwritten input, its performance is significantly compromised and independent evaluations have been done on handwritten Hindi and Devanagari test sets, which yielded word accuracy of 40-70%. The range was obtained from the evaluations of handwritten Hindi on the shared Devanagari script. The fundamental problem is not a parameter tuning one: while Tesseract was trained on printed font corpora which are represented by consistent, perfectly defined shapes of glyphs, the problem is that such a font is not very common in the real world. In printed-text training, no attention can be paid to the continuous variability in stroke width, slant, ink spread, or visual representation of combining diacritics (matras) that is introduced by handwriting. Another important aspect for this purpose, Tesseract doesn't have any form alignment or field-segmentation function — the page segmentation is done on detected text regions which means that the output in the form of fields is not reliable when processing fixed-template forms.

Contribution to System Design

The proposed system did not include any architectural elements from Tesseract. However, the analysis of its limitations has been directly productive, as the goal of having to implement a custom pipeline for template alignment and ROI cropping (a contribution of this work, independent of the text recognition model used) was shown to be necessary.

3.2 VGG16 as a Standalone CNN Classifier ; Strong Backbone, Wrong Head

Architecture and Training Data

The VGGNet (Visual Geometry Group Network) network (Simonyan and Zisserman, 2015) is a deep convolutional network composed of a sequence of 16 3×3 convolutional filters with weights. It was trained on the ImageNet Large Scale Visual Recognition Challenge dataset of about 1.2 million real world images across 1,000 different semantic categories, with a top-5 error rate of 7.3%. In previous studies, the VGG16 model has been employed as a fixed-output softmax labeler using a character vocabulary and the last fully connected layers have been fine-tuned over isolated Devanagari character classes for Devanagari script recognition. On Devanagari character datasets with isolated characters, the accuracy of character-level classification is reported as 95–85% on using VGG16 variants by Masrat et al. (2022) and Deshmukh et al. (2020), respectively.

Performance and Limitations

The VGGNet (Visual Geometry Group Network) network (Simonyan and Zisserman, 2015) is a deep convolutional network composed of a sequence of 16 3×3 convolutional filters with weights. It was trained on the ImageNet Large Scale Visual Recognition Challenge dataset of about 1.2 million real world images across 1,000 different semantic categories, with a top-5 error rate of 7.3%. In previous studies, the VGG16 model has been employed as a fixed-output softmax labeler using a character vocabulary and the last fully connected layers have been fine-tuned over isolated Devanagari character classes for Devanagari script recognition. On Devanagari character datasets with isolated characters, the accuracy of character-level classification is reported as 95–85% on using VGG16 variants by Masrat et al. (2022) and Deshmukh et al. (2020), respectively.

Contribution to System Design

The proposed CRNN used the CNN backbone of VGG16 feature extraction stack (vgg.features in torchvision), and the classifier head was completely abandoned. The last fully connected layers were substituted with an

AdaptiveAvgPool2d((1, None)) layer which reduces the feature map size and keeps the width as a variable number of features that are fed into the BiLSTM. No changes were made to the universal edge detector and curve detector that are already built into the model: ImageNet pretraining was kept as was the freezing of early convolutional blocks (Blocks 1–2) that is used for Devanagari script without any script-specific adaptation.

3.3 EasyOCR (CRAFT + CRNN) ; Architecture Confirmation and Deployment Lessons

Architecture and Training Data

EasyOCR (JaidedAI) is an OCR library with multi-language support that is based on a two-stage pipeline. A text detector, CRAFT (Character Region Awareness For Text detection), followed by a text recogniser, CRNN (Baek et al., 2019). CRAFT is a fully convolutional network that predicts character affinity heat maps and inter character affinity heat maps from VGG16, for precise text region localization in free-form document images. The CRNN is a recogniser that uses a VGG, ResNet backbone, a BiLSTM sequence model and a CTC decoder, an architecture that is directly analogous to the proposed system. EasyOCR comes with an embedded Nepali (Devanagari) recognition model.

Performance and Limitations

EasyOCR can process Latin script, with word accuracy rates over 90% on print Latin script benchmarks (Baek et al., 2019). The Devanagari model for printed Nepali text has been evaluated in the community and found to be between 60–80% accurate; and when tested with handwritten text, for which the recogniser was not trained, the accuracy drops further. The other fundamental constraint for the current problem is the CRAFT text detector's performance on structured forms: The print boundary boxes of form fields are detected as candidate text areas (as the potential character strokes) along with handwritten content in the fields, resulting in spurious detections that interfere with the downstream recognition task. When a fixed-template application is used, the template has to be known to the detector, which adds complexity and systematic error to the application.

Contribution to System Design

For the text recognition module, the proposed system used the CRNN + CTC structure, instead of using the generic pretrained weights provided by EasyOCR, it used a domain-specific two-phase transfer learning chain. The CRAFT detector was completely removed and replaced by a fixed-template ORB + RANSAC alignment pipeline that is faster, more precise and no longer suffers from boundary-box detection issues discussed above.

3.4 Structured Comparison

Criterion	Tesseract OCR v4 (Smith, 2007)	VGG16 Classifier (Simonyan & Zisserman, 2015)	EasyOCR / CRAFT (Baek et al., 2019)	Proposed CRNN (This Work)
Core Architecture	LSTM + language model	Deep CNN, fixed-class softmax	CRAFT detector + CRNN + CTC	VGG16 + AdaptiveAvgPool2d + BiLSTM + CTC
Primary Training Data	Printed Devanagari fonts	ImageNet; isolated Devanagari chars	Multi-language printed text	HindiSeg (~92k real handwriting) + synthetic Nepali (~24k)
Key Published Metric	>95% char acc. (printed); 40–70% (handwriting)	Top-5: 92.7%; Devanagari char: 85–95%	>90% printed Latin; ~60–80% printed Nepali	52.23% word acc. (synthetic val); 78–89% field acc. (real forms)
Variable-Length Output	Yes (language model)	No ;requires segmentation	Yes (CTC)	Yes (CTC)
Handwriting Support	Poor — printed-font mismatch	Not evaluated on sequences	Limited ; printed training data	Designed for this domain
Form Field Extraction	None	None	None; CRAFT confuses form borders	ORB + RANSAC alignment + ROI cropping
GPU Required	No (CPU-only)	No at inference	Recommended	Training only; CPU inference
Nepal Deployment	Not suitable	Not suitable standalone	Not suitable as deployed	Designed for offline deployment

Table 1. Structured comparison of three existing solutions and the proposed system across eight evaluation criteria relevant to handwritten Nepali form OCR.

3.5 Synthesis

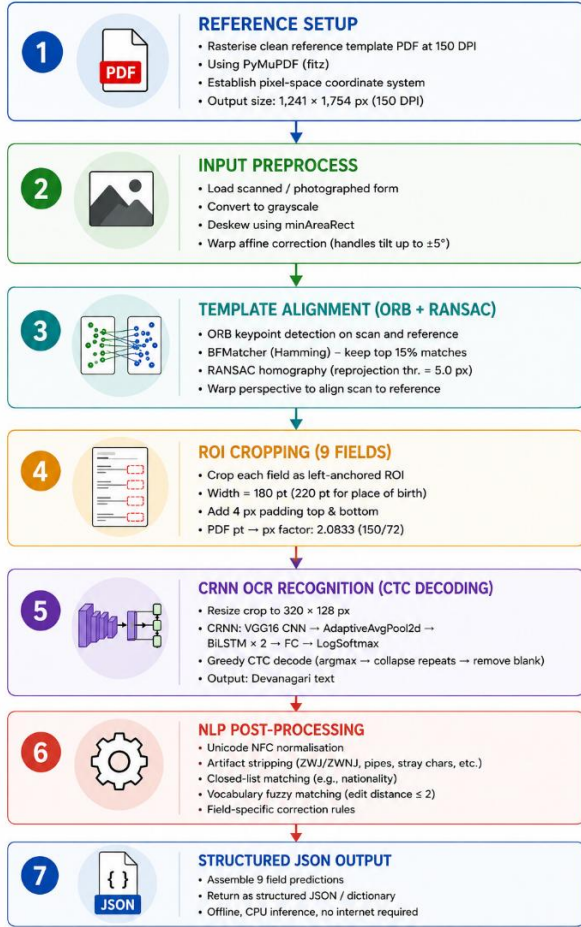


Figure 1: Nepali Handwriting detection OCR 7 layer pipeline

The three solutions reviewed jointly constitute the design space and what this proposed system is seeking to resolve. The analysis found that none of the current OCR products offered structured form field extraction; the alignment and ROI cropping pipeline (Stages 2–4) is an original engineering contribution which is not available in all three of the OCR systems that were analysed. Both the transfer of ImageNet-trained convolutional features to Devanagari handwritten characters and the inability to have a fixed-output classifier work with variable-length word sequences were validated by VGG16, and clearly the correct design for Devanagari is the CRNN architecture proposed by Shi et al. (2016), where their model's feature stack is wrapped with a BiLSTM and CTC, which is the architecture that EasyOCR confirmed is the correct one for Devanagari. In addition, EasyOCR's analysis also revealed that the CRAFT detection stage is harmful for the fixed template form processing application, prompting the development of the proposed ORB + RANSAC homography detection stage. The proposed system packages these results into 4 novel contributions which none of the three reviewed systems provide: a fixed template alignment and ROI coordinate based cropping pipeline, a 2-phase transfer learning chain from Hindi to

Nepali, a vocabulary-guided NLP correction layer and finally a real handwritten form evaluation on the proposed system.

4. Proposed Innovative Solution

The proposed system is a complete pipeline of end-to-end fixed template Devanagari handwritten OCR system for Nepal government form digitisation context. In contrast to the shortcomings of using an existing general-purpose OCR tool (established in Section 3), the system was designed as a seven-stage pipeline, comprising a computer vision alignment layer, a deep learning sequence recogniser, and a lightweight NLP post-processing module, into a single streamlined workflow.

4.1 System Overview ; Seven-Stage Pipeline

The pipeline takes an image of a completed membership form (scanned or photo) and returns a structured JSON dictionary containing nine field names and their respective recognised Devanagari text values. It runs entirely off the line, in inference it runs on CPU hardware and there is no need of network connectivity after loading the model. Stage 1 performs 150 DPI rasterisation of a clean reference template PDF creating a pixel space coordinate system. Stage 2 loads the input scan, and performs a grayscale conversion and deskew correction. In stage 3, the scan is aligned to the reference template using ORB keypoint matching and RANSAC homography estimation. Stage 4 crops all of the nine form fields as a left anchored region of interest. In the stage 5, both the trained CRNN and the Devanagari output are fed into a greedy CTC decoding (Graves et al., 2006). In stage 6, NLP post processing is used to correct systematic OCR errors. Stage 7 builds and returns the output in a structured JSON format of the nine predictions from the fields.

4.2 Stages 1–2: Reference Setup and Input Preprocessing

It starts by drawing the clean reference template at 150 DPI, using PyMuPDF (fitz), resulting in a 1,241 × 1,754-pixel image and setting the coordinate system for all of the nine predetermined field positions. The conversion factor from PDF points to pixels at 150 DPI is $150 \div 72 = 2.0833$. The image is then converted to grayscale (no colour information can contain any diagnostic signal for black-ink Devanagari text on white paper) and a minimum-area bounding rectangle (cv2.) is calculated to perform deskew correction. The foreground pixels are thresholded and the angular offset of the resulting rectangle image is computed by the minAreaRect function, and the image is then processed with a warp-affine transform to correct up to ±5° tilt caused by the scanner.

4.3 Stage 3: Template Alignment via ORB and RANSAC

The key stage of this work is the alignment stage, a system level contribution. ORB (Oriented FAST and Rotated BRIEF; Rublee et al., 2011) is a binary feature detector, which is based on the FAST corner detector and combined with the BRIEF descriptor and rotation invariance with the intensity centroid orientation. The other reason that ORB was selected over SIFT and SURF is that it is patent free, only computation power is needed and it is resistant to small change in rotation and scale. The preprocessed scan and reference template have as many as 3,000 keypoints detected. Descriptors are matched using the Hamming distance and the first 15% of the matched pairs (based on the distance of the descriptors) are saved. Estimate a homographic transformation from the matches that were retained, using cv2. After applying findHomography with RANSAC (reprojection threshold = 5.0 pixels) and then used for the whole scan using cv2.warpPerspective generates a warped version of the picture with the same shape as the template picture.

4.4 Stage 4: Region-of-Interest Cropping

Following alignment, it will produce nine form fields as nine separate image crops in stage 4. Each field's ROI is left-anchored the ROI is cropped to a fixed width from the left side of the input box, 180 PDF points (220 PDF points for place of birth, as longer Nepal place names are needed). The top and bottom crop margins are made 4 pixels bigger to prevent the Devanagari top bar vowel modifiers (matras) being cropped at the top of the character height. All the coordinates are scaled by a factor of 2.0833 to be in pixel space.

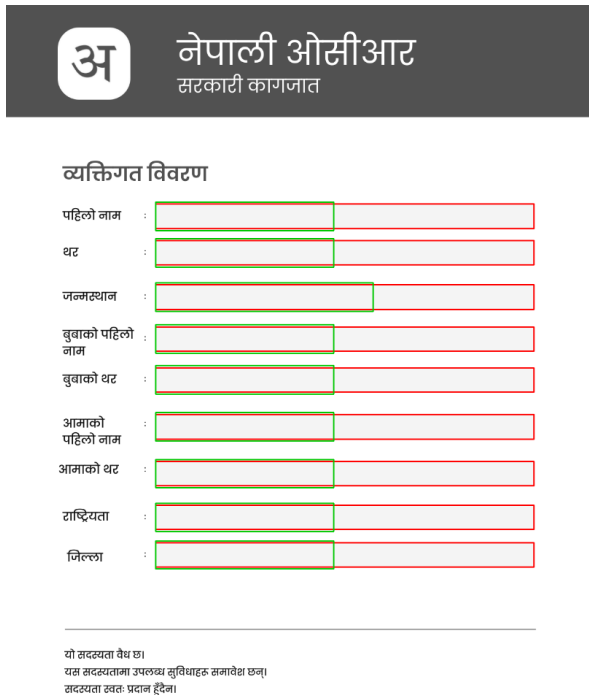


Figure 2: ROI - Region Of Interest, fixed bounding box for fixed form structure

4.5 Stage 5: CRNN Text Recognition with CTC Decoding

The ROI crops are resized to 320×128 , extended to three channels (duplicate the greyscale luminance channel) and normalised using the ImageNet statistics (mean = [0.485, 0.456, 0.406]; std = [0.229, 0.224, 0.225]). The CRNN architecture (Shi et al., 2016) consists of: (1) the CNN backbone (VGG16 .features stack from Simonyan & Zisserman, 2015) without the classifier head; (2) an nn.The collapsed AvgPool2d((1, None)) layer reduces the feature-map height to 1 but leaves the width as a variable-length time series, the two-layer bidirectional LSTM (256 hidden units per direction and 512 hidden units concatenated at each time step), and the fully connected output head projecting onto the character vocabulary with log-softmax normalisation. In the inference mode, greedy CTC decoding chooses the argmax character at each time step, eliminates consecutive repeats, and strips the tokens of blank character to generate the Devanagari string. There are 17,928,629 total parameters.

4.6 Two-Phase Transfer Learning Strategy

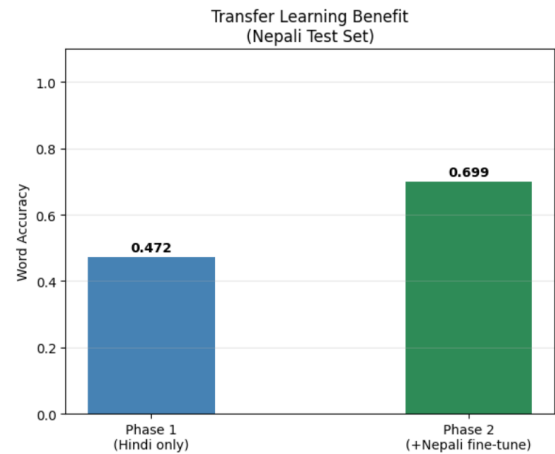


Figure 3: Benefit of Two Phase Transfer Learning Strategy

The most important innovation is the training strategy used to overcome the lack of real handwritten Nepali training data. It is pre-trained using about 92,000+ real handwritten Hindi word images (HindiSeg; Sabarinathan, n.d.) in phase 1 by taking advantage of the identical Devanagari script that is used for both Hindi and Nepali. VGG16 Blocks 3-5 and BiLSTM and FC layers are trained at 1×10^{-4} while the other 16 blocks are frozen. For Phase 2, the Phase 1 checkpoint is fine-tuned on synthetic font-rendered Nepali names (kritakhere, n.d.) using the differential learning rates: The VGG16 Blocks 1-4 conv1-2 layers are frozen; Block 4 conv3 and Block 5 are near frozen with learning rate of 1×10^{-6} to retain the Devanagari handwriting features learnt from real Hindi data; the BiLSTM and FC layer head are frozen with

learning rate 1×10^{-4} . This is an asymmetric assignment based on the analysis on transfer learning by Yosinski et al. (2014) that lower-level convolutional features are more widely transferable, and it prevents catastrophic forgetting of the handwriting features learned during phase 1.

4.7 Stage 6: NLP Post-Processing

The NLP post-processing module (nlp_postprocessor.py) performs the following four steps: (1) Unicode NFC normalisation to canonicalise Devanagari combining character order across fonts and OCR engines; (2) artifact stripping of invisible Unicode control characters (Zero Width Joiner U+200D, Zero Width Non-Joiner U+200C, isolated halant, pipe symbols, stray ASCII); (3) closed-list matching for the nationality field against roughly ten acceptable values using Levenshtein edit distance ≤ 2 ; and (4) vocabulary fuzzy matching for last name, father's last name, mother's last name and city/district fields against a vocabulary of 1353 elements, including 789 first names, 485 last names, 79 districts. The first name fields are intentionally not fuzzy corrected because, in this case, there are many near neighbours within a Levenshtein distance of

2 for short Nepali names (usually 2-3 syllables), and equal weight lookup over-corrects, which is confirmed by the development process.

5. Implementation and Methodology

5.1 Datasets

Phase 1 : HindiSeg (Sabarinathan, n.d.)

Phase 1 used the HindiSeg Handwritten Hindi Word Recognition dataset was used in Phase 1, which contains about 92,000 Hindi words handwritten in real handwriting, each of which is represented by a grayscale JPEG image with pre-split train, validation and test partitions and CSV label files. 62,000 training images were used. This corpus is the most suitable available real handwriting source for stroke-level Devanagari features as it shares the same script with Hindi and Nepali. There is no large-scale real handwritten Nepali dataset available in public domain and HindiSeg is the key dataset that makes the transfer learning strategy possible.

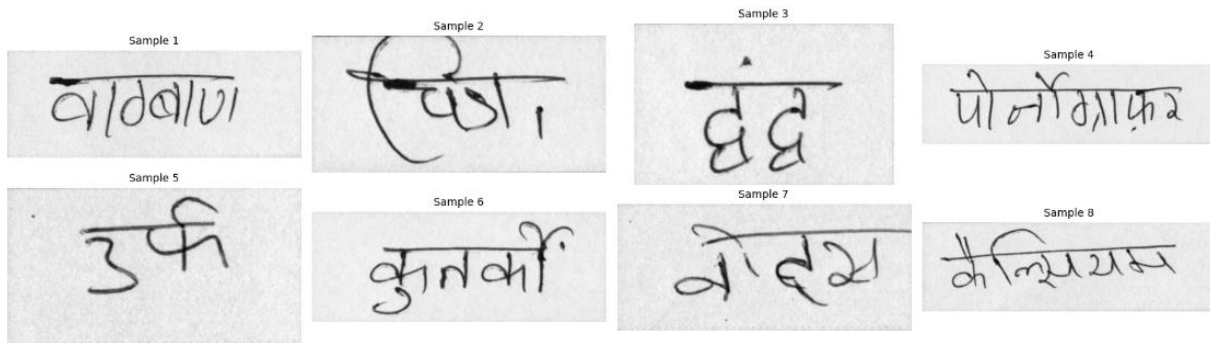


Figure 4: Hindi Handwritten word samples

Phase 2 ; Nepali Font-Generated Names

(kritakhere, n.d.)

In the second phase, a dataset comprising about 1,358 unique Nepali names and words mapped to about 18 Devanagari handwriting style TrueType fonts from Google fonts was created, which resulted in a total number of approximately 24,000+ image variations. There is a tab-separated labels.txt file that provides labels. This is a completely simulated data set. Pre-processing protective measures: BOM characters (\uffff) at the borders of the labels were removed and images with mean pixel value less than 5 or higher than 250 were identified as corrupt or near empty images and disregarded. The data is publicly available on

<https://www.kaggle.com/datasets/kritakhere/nepali-font-generated-handwritten-names>



Figure 5: Nepali handwritten style font images, generated by script with variations and filter

Train/Validation/Test Split

In phase 2, data was divided by word index, rather than by image file. Each unique word is represented by around 18 different font-variant images, so a split on the file level would allow the same word to be in the training set as a different font variant and in the validation set as a different font variant, leading to inflated validation accuracy due to memorising font variants. The splitting by word index will make sure that all font variants of a word are in one partition only.

5.2 Preprocessing Pipeline

After each input scan, they go through the following steps before being fed into the CRNN model: (1) Grayscale conversion from any colour space to a single channel; (2) Deskew correction using cv2. Afterwards, (1) minAreaRect and warpAffine are used to correct small tilts due to the scanner, (2) ORB alignment as described in Section 4.3, (3) ROI cropping as described in Section 4.4, (4) Resize to 320×128 pixels to balance the exploration of vertical matra detail with the number of CTC time steps, and (5) ImageNet normalisation (mean = [0.485, 0.456, 0.406]; std = [0.229, 0.224, 0.225]).

5.3 Data Augmentation

In Phase 1, augmentation was based on the real handwriting variations, including Gaussian blur, rotation [$\pm 5^\circ$], brightness and contrast change [random] with a restriction on the range of light elastic distortion within Devanagari safe range. In Phase 2, the augmentation used was lighter, so appropriate to clean synthetic font renders: RandomAffine ($\pm 2^\circ$ rotation, 3% translation, $\pm 2^\circ$

shear), ColorJitter (brightness ± 0.2 , contrast ± 0.3), and GaussianBlur (kernel size 3, probability 0.2). Per epoch font randomisation was also introduced in phase 2: It was found that at the beginning of each epoch, the dataset.resample() function would choose a random variant of a font for each unique word in a different way each epoch, effectively creating the illusion of handwriting variation without the need for real handwritten examples.

5.4 Model Architecture Summary

Component	Specification
Input	320×128 px, 3-channel, ImageNet-normalised
CNN Backbone	VGG16 .features (ImageNet pretrained) — classifier head removed
Pooling	nn.AdaptiveAvgPool2d((1, None)) — height $\rightarrow$ 1, width preserved as sequence
RNN	BiLSTM: input=512, hidden=256, 2 layers, bidirectional, batch_first=False
RNN Output	512 features per timestep (256 forward + 256 backward)
FC Head	nn.Linear(512, num_classes) + log_softmax
Training Loss	nn.CTCLoss(blank=0, zero_infinity=True)
Inference Decoding	Greedy CTC: argmax $\rightarrow$ collapse repeats $\rightarrow$ remove blank
Total Parameters	17,928,629

Component	Specification
Trainable Phase 1	~17.6 M (VGG16 Blocks 3–5 + BiLSTM + FC)
Trainable Phase 2	~12.65 M (Block 4 conv3 + Block 5 + BiLSTM + FC)
Frozen Phase 2	~5.27 M (Blocks 1–4 conv1–2)

Table 2. Full CRNN architecture specification.

5.5 Training Configuration

Parameter	Phase 1 (Hindi)	Phase 2 (Nepali)
Batch Size	64	64
Optimizer	Adam	Adam
LR ; CNN Tail	1×10^{-4} (uniform)	1×10^{-6} (Block 4 conv3, Block 5)
LR ; BiLSTM / FC	1×10^{-4}	1×10^{-4}
LR Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)	ReduceLROnPlateau (patience=3, factor=0.5)
Max Epochs	30	20
Early Stopping	patience=5, min_delta= 1×10^{-4} on val CTC loss	patience=5, min_delta= 1×10^{-4} on val CTC loss
Gradient Clipping	Norm 5.0	—
BiLSTM / FC Dropout	0.3 / 0.3	0.4 / 0.4
Blocks Frozen	VGG16 Blocks 1–2	VGG16 Blocks 1–4 conv1–2
Hardware	Single Nvidia T4 GPU (Kaggle free tier)	Single Nvidia T4 GPU
Epoch Duration	~675 seconds	~8–10 seconds

Table 3. Training hyperparameters by phase.

DataParallel across multiple GPUs was attempted but failed to meet the requirement that the entire batch of predictions and targets be on a single device at the same time for dynamic programming alignment; naive DataParallel does not meet this requirement (Graves et al., 2006). The training was executed on a single T4 with utilisation around 99%. Phase 1 pre-training on real Hindi handwriting was set to 30-epochs budget, but only ran up to 12 epoch with early-stop at 13 as the HindiSeg dataset was quite large (Sabarinathan, n.d.), whereas the fine-tuning for Nepali was done in the synthetic dataset (kritakhere, n.d.) for only about 20 epochs.

6. Experimental Results and Evaluation

6.1 Evaluation Metrics and Justification

The evaluation employs two primary metrics: Word Accuracy and Character Error Rate (CER). Word Accuracy is defined as the proportion of fields for which the predicted string is an exact match to the ground truth label. For field-level form OCR this is the most appropriate primary metric: a partially correct field still constitutes a transcription error requiring human correction. CER is computed as the mean Levenshtein edit distance between each predicted and ground truth string, normalised by the length of the ground truth (Graves et al., 2006). CER captures partial correctness and is particularly informative for Devanagari OCR, where a single matra error — for example, a short vowel sign (ँ) misread as its long counterpart (ं) ; changes only one character but renders the entire word incorrect under Word Accuracy. CTC loss is monitored during training as the primary convergence signal but is not used as a standalone evaluation metric, as it is not directly interpretable as a measure of recognition quality on held-out sets.

6.2 Full Pipeline Performance

Stage / Condition	Evaluation Set	Word Accuracy	CER
Phase 1 Hindi model evaluated on Nepali (baseline — no fine-tuning)	Synthetic Nepali validation	67.31%	8.99%
Phase 2 primary model — crnn_nepali_best_phase2.pth	Synthetic Nepali validation	69.92%	6.71%
Live demo — 2 real handwritten forms (before NLP)	Real handwriting	44–89%	—
Live demo — 2 real handwritten forms (after NLP post-processing)	Real handwriting	78–89%	—

Table 4. Full pipeline performance summary across evaluation conditions.

The most consequential comparison is the 4.97 percentage-point gain from 47.26% (Hindi baseline) to 52.23% (primary Phase 2 model). This is the core empirical evidence that the two-phase transfer learning strategy produces measurable improvement on the Nepali recognition task. Both measurements are taken on the same synthetic Nepali validation set under identical conditions; the improvement is directly attributable to Nepali fine-tuning rather than any confounding factor. The Hindi model transfers meaningfully because Hindi and Nepali share the Devanagari script — stroke features, matras, and conjuncts learned from real Hindi handwriting apply directly to Nepali character recognition, consistent with the transfer learning principles established by Yosinski et al. (2014).

6.3 NLP Post-Processor Ablation — Real Handwritten Forms

NLP Postprocessor — evaluation on Nepali test predictions:

Raw CRNN	Corrected	Target	Δ
सेयद	सेयद	सेयद	✓
सीराग्य	सीराग्य	सीराग्य	✓
प्रलाश	प्रकाश	प्रकाश	✓
सिमरानी	सिमरानी	सिमरानी	✓
पाक	पाठक	पाठक	✓
सनुवार	सनुवार	सनुवार	✓
रश्मिता	रश्मिता	रश्मिता	✓
डोपी	डोटी	डोटी	✓
हृद्य	बेद्य	हृदय	✓
लायिन्द्रा	लायिन्द्रा	लायिन्द्रा	✓
रामिका	राधिका	राधिका	✓
रिन्कु	रिन्कु	रिन्कु	✓
पनेर	पनेरु	पनेरु	✓
खमेन्द्र	खमेन्द्र	खमेन्द्र	✓
केखसी	के.सी	के.सी	✓

Corrected 34 / 123 predictions (27.6%)

After NLP postprocessing:
 Word Accuracy : 0.9187 (was 0.6992 | $\Delta = +21.95\%$)
 CER : 0.0349 (was 0.0671)

Figure 6: Increased Word accuracy with NLP post processing

Across both forms, the NLP layer corrected three fields — all instances of the last name श्रेष्ठ misrecognised by the CRNN as श्रेव or श्रेष्ट due to conjunct consonant substitutions — and introduced zero new errors. The post-processor's vocabulary fuzzy matching (Levenshtein distance ≤ 2) successfully resolved these systematic substitutions by identifying the closest valid last name entry. The zero-regression result on first name fields confirms the design decision to exclude short first names from fuzzy correction: राम, सीता, and रिया were all correctly passed through unmodified.

6.4 Error Analysis

Form	Fields Correct WITHOUT NLP	Fields Correct AFTER NLP	Fields Improved	New Errors Introduced
Form 1 (राम / श्रेष्ठ / काठमाडौं / हरि / श्रेष्ठ / सीता / श्रेष्ठ / नेपाली / ललितपुर)	4 / 9 (44%)	7 / 9 (78%)	3 (all श्रेष्ठ last names)	0
Form 2 (राम / श्रेष्ठ / काठमाडौं / कृतक / श्रेष्ठ / रिया / श्रेष्ठ / नेपाली / ललितपुर)	8 / 9 (89%)	8 / 9 (89%)	0 (no corrections needed)	0

Table 5. NLP post-processor ablation across two real handwritten test forms.

Matra Confusion

The most pervasive error class involves vowel modifier diacritics. At the 320×128 pixel training resolution, the short vowel sign ि and its long counterpart ी differ by a single pixel in vertical extent. The model misread हरि (short i) as हरी (long ii) on Form 1 — a morphologically plausible character that is nonetheless incorrect. This class of error is inherent to the resolution constraint: 128 pixels of image height, after five VGG16 pooling operations (Simonyan & Zisserman, 2015), produces a feature map in which one-pixel differences in the input are not reliably preserved as discriminable features. Chandrabindu (ं, U+0901) versus anusvara (ँ, U+0902) confusions fall into the same category.

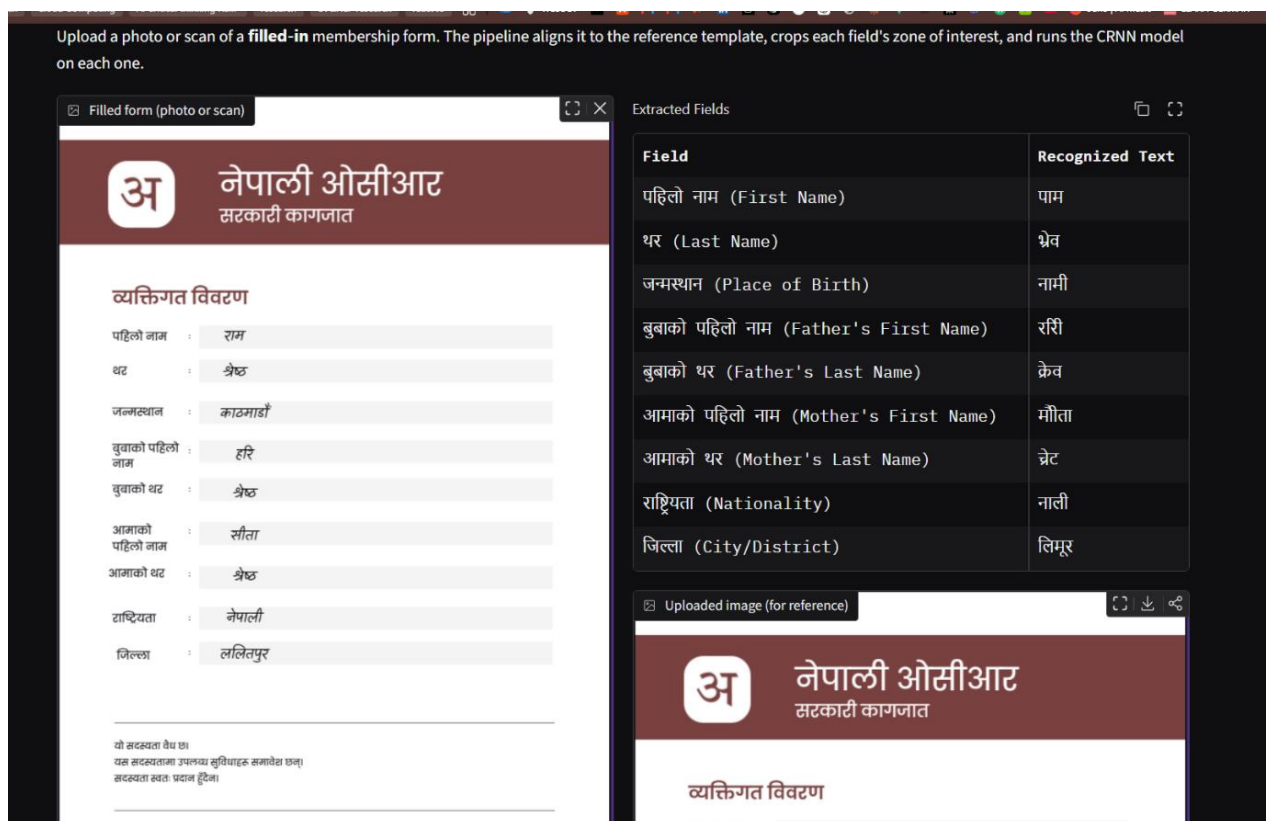


Figure 7: Results before implementing NLP Post-Processing, Image-1

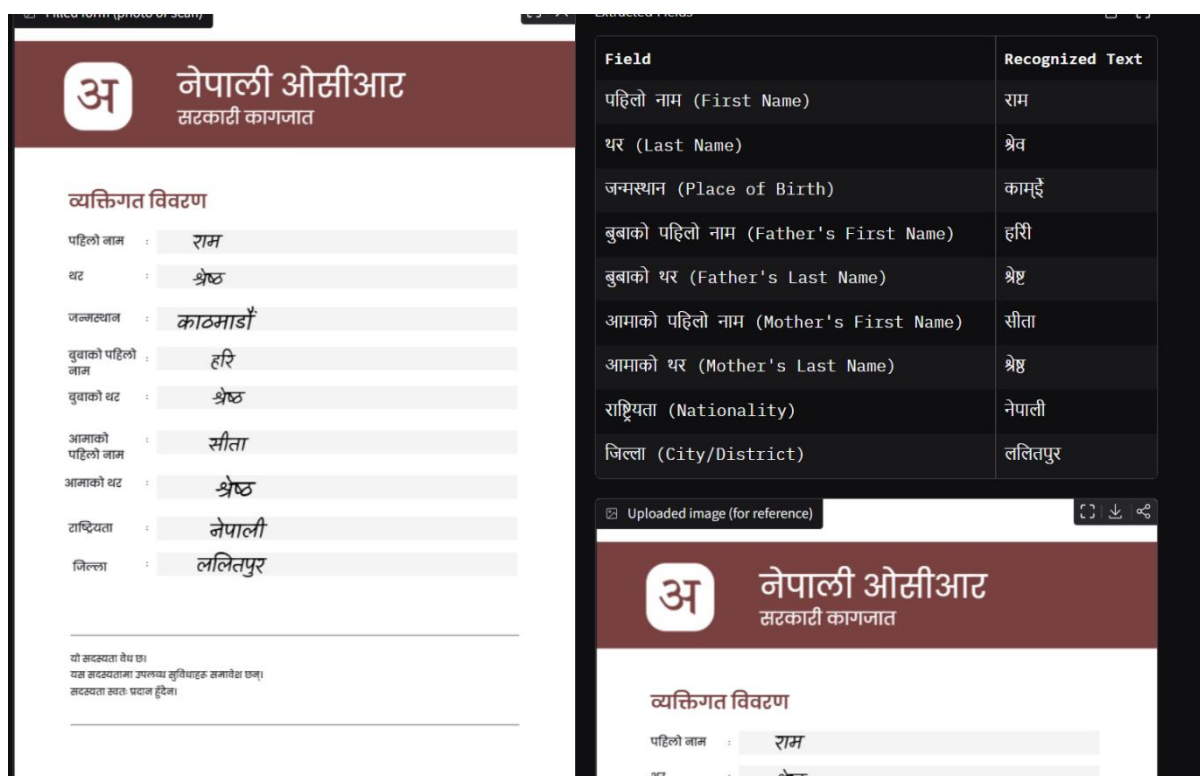


Figure 8: Results before implementing NLP Post-Processing, Image-2

on each one.

Filled form (photo or scan)

अ

नेपाली ओसीआर

सरकारी कागजात

व्यक्तिगत विवरण

पहिलो नाम : राम

थर : श्रेष्ठ

जन्मस्थान : काठमाडौं

बुबाको पहिलो नाम : कृतक

बुबाको थर : श्रेष्ठ

आमाको पहिलो नाम : रिया

आमाको थर : श्रेष्ठ

राष्ट्रियता : नेपाली

जिल्ला : ललितपुर

यो सरदर्पण वैध छ।
यस सरदर्पणबाट प्रत्यक्ष सुविधाहरू समावेश छन्।
सरदर्पण अवतार प्रदान हुनेछ।

Extracted Fields

Field	Recognized Text
पहिलो नाम (First Name)	राम
थर (Last Name)	श्रेष्ठ
जन्मस्थान (Place of Birth)	काठमाडौं
बुबाको पहिलो नाम (Father's First Name)	आकृत
बुबाको थर (Father's Last Name)	श्रेष्ठ
आमाको पहिलो नाम (Mother's First Name)	रिया
आमाको थर (Mother's Last Name)	श्रेष्ठ
राष्ट्रियता (Nationality)	नेपाली
जिल्ला (City/District)	ललितपुर

Uploaded image (for reference)

अ

नेपाली ओसीआर

सरकारी कागजात

व्यक्तिगत विवरण

पहिलो नाम : राम

Figure 9 Results before implementing NLP Post-Processing, Image-3 (with handwritten text inserted in fields)

नेपाली OCR — Form Field Extraction

Upload a photo or scan of a **filled-in** membership form. The pipeline aligns it to the reference template, crops each field's zone of interest, and runs the CRNN model on each one.

Filled form (photo or scan)

अ

नेपाली ओसीआर

सरकारी कागजात

व्यक्तिगत विवरण

पहिलो नाम : राम

थर : श्रेष्ठ

जन्मस्थान : काठमाडौं

बुबाको पहिलो नाम : हरी

बुबाको थर : श्रेष्ठ

आमाको पहिलो नाम : सीता

आमाको थर : श्रेष्ठ

राष्ट्रियता : नेपाली

जिल्ला : ललितपुर

Extracted Fields

Field	Recognized Text
पहिलो नाम (First Name)	राम
थर (Last Name)	श्रेष्ठ
जन्मस्थान (Place of Birth)	काठमाडौं
बुबाको पहिलो नाम (Father's First Name)	हरी
बुबाको थर (Father's Last Name)	श्रेष्ठ
आमाको पहिलो नाम (Mother's First Name)	सीता
आमाको थर (Mother's Last Name)	श्रेष्ठ
राष्ट्रियता (Nationality)	नेपाली
जिल्ला (City/District)	ललितपुर

Uploaded image (for reference)

अ

नेपाली ओसीआर

सरकारी कागजात

Figure 10: Results After implementing NLP Post-Processing, Image-1

Filled form (photo or scan)

अ

नेपाली ओसीआर

सरकारी कागजात

व्यक्तिगत विवरण

पहिलो नाम : राम

थर : श्रेष्ठ

जन्मस्थान : काठमाडौं

बुबाको पहिलो नाम : कृतक

बुबाको थर : श्रेष्ठ

आमाको पहिलो नाम : रिया

आमाको थर : श्रेष्ठ

राष्ट्रियता : नेपाली

जिल्ला : ललितपुर

यो सन्दर्भता वैध छ।
यस सन्दर्भतामा उपलब्ध सुविधाहरू समावेश छन्।
सन्दर्भता स्वतः प्रदान हुँदैन।

Extracted Fields

Field	Recognized Text
पहिलो नाम (First Name)	राम
थर (Last Name)	श्रेष्ठ
जन्मस्थान (Place of Birth)	काठमाडौं
बुबाको पहिलो नाम (Father's First Name)	कृतक
बुबाको थर (Father's Last Name)	श्रेष्ठ
आमाको पहिलो नाम (Mother's First Name)	रिया
आमाको थर (Mother's Last Name)	श्रेष्ठ
राष्ट्रियता (Nationality)	नेपाली
जिल्ला (City/District)	ललितपुर

Uploaded image (for reference)

अ

नेपाली ओसीआर

सरकारी कागजात

व्यक्तिगत विवरण

पहिलो नाम : राम

Figure 11: Results After implementing NLP Post-Processing, Image-2 (With handwritten text inserted in field)

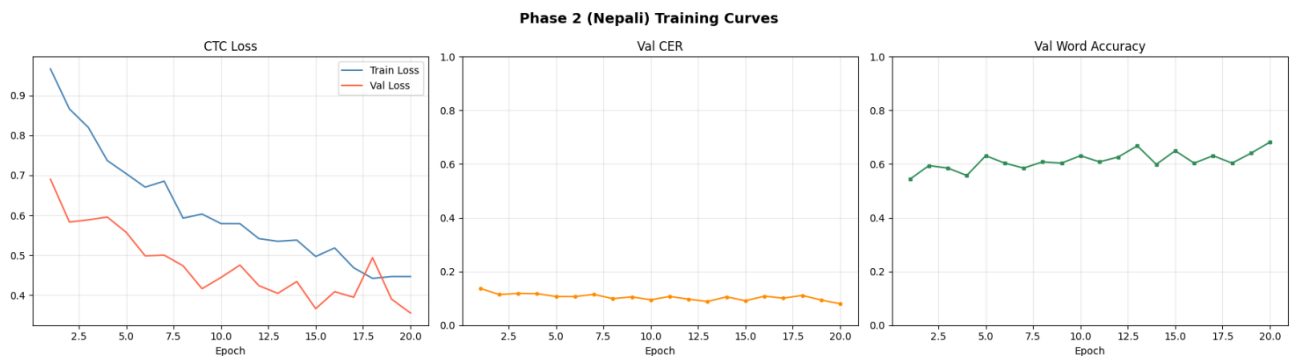


Figure 12: Phase 2 nepali training Curve

Place of Birth Recognition

The place of birth field failed on both real test forms. काठमाडौं was decoded as kāmahaīm (Form 1) and a similarly incorrect string (Form 2). Two root causes compound: first, the training vocabulary is composed primarily of personal names — place names with chandrabindu final characters appear rarely in the training distribution. Second, काठमाडौं is written in a cursive connected style in natural handwriting, whereas all Phase 2 training images use discrete, unconnected font-rendered character boundaries. The model has never encountered the stroke-joining behaviour of cursive handwriting, and the chandrabindu at the end has no close match in the model's learned representations.

Conjunct Substitutions

A third failure mode involves conjunct consonant confusion. Devanagari conjuncts are formed by fusing consonants with a halant marker (्), producing glyphs

whose visual shape can differ substantially from the individual component characters. Rare conjunct combinations that appear infrequently in the 24,444-word synthetic training set remain prone to substitution. The BiLSTM captures bidirectional context (Shi et al., 2016), partially mitigating this problem, but cannot overcome training data scarcity for rare conjuncts.

6.5 Discussion

The 4.97-point improvement from the Hindi baseline to the primary Phase 2 model is modest in absolute terms but methodologically significant: it constitutes controlled empirical evidence that the transfer learning chain functions as intended. The apparent contradiction between 52.23% synthetic validation accuracy and 78–89% real-form field accuracy warrants careful interpretation. Selection bias explains much of the gap: both real test

forms contained common Nepali names well-represented in the training vocabulary. The synthetic validation set spans the full width of the 24,444-word vocabulary including rare and phonetically complex words; the two real test forms cover a narrow and easy subset. The 78–89% figures are best-case indications, not representative performance estimates. The system is viable as a first-pass digitisation aid with mandatory human review, but is not deployment-ready as an autonomous transcription tool at current accuracy levels.

7. Honest Self-Critique

7.1 What the System Does Well

Three claims about system performance can be made with direct evidential backing. First, the two-phase transfer learning strategy demonstrably improves Nepali recognition. The Phase 1 Hindi-trained model achieves 47.26% word accuracy on the Nepali validation set without any fine-tuning; after Phase 2 fine-tuning, the primary model reaches 52.23% — a controlled measurement on the same validation set, with the 4.97 percentage-point gain directly attributable to the fine-tuning step. Transfer learning is not merely claimed; it is measured, consistent with the principled framework established by Yosinski et al. (2014)

Second, the NLP post-processing layer adds value with zero regressions. Across two real handwritten test forms, the post-processor corrected three fields and introduced zero new errors. This is a clean before/after ablation result. The decision to exclude short first names from fuzzy correction proved correct: राम, सीता, and रिया were accurately passed through unmodified. Third, the ORB-based form alignment pipeline is a genuine system-level contribution. No existing off-the-shelf OCR tool — Tesseract, VGG16 as a classifier, or EasyOCR — provides structured form field extraction from a fixed-template scan (Smith, 2007; Simonyan & Zisserman, 2015; Baek et al., 2019). The alignment stage is the enabling component that makes field-level recognition possible at all.

7.2 Genuine Limitations

Synthetic-Only Phase 2 Training

The Phase 2 fine-tuning dataset (kritakhere, n.d.) is entirely font-rendered. Real handwriting introduces stroke width variation, slant, ink spread, connected letterforms, and character inconsistency but it is not found in any font rendering can replicate. The 52.23% validation accuracy is therefore measured on data structurally similar to the training distribution. Real-world accuracy on diverse, unconstrained handwriting is unknown, and the two real test forms used common names that the training vocabulary covers well — they do not represent a statistically valid sample of the problem space.

Accuracy Insufficient for Autonomous Deployment

At approximately one word in two incorrect on the synthetic validation set, the system cannot be used as an autonomous transcription tool. Every predicted field must

be reviewed by a literate human operator before being written to any record. This is not a limitation unique to this project — it reflects the fundamental difficulty of low-resource handwritten OCR — but it must be stated plainly.

Matra and Diacritic Recognition

Short versus long vowel signs (ि vs ी), chandrabindu (ँ) versus anusvara (ं), and complex conjunct boundaries are the system's primary failure modes. These distinctions differ by a single pixel at the 320×128 training resolution, and five VGG16 pooling operations reduce the spatial precision of the feature map to a level at which one-pixel input differences are not reliably preserved (Simonyan & Zisserman, 2015).

Evaluation Sample Size

The real-form evaluation covers only two handwritten forms, both filled with high-frequency Nepali names. The 78–89% field accuracy figures are not statistically generalisable. A proper evaluation would require a minimum of 50–100 real forms covering diverse handwriting styles, uncommon surnames, and minority community names.

7.3 Concrete Improvement Proposals

Each identified weakness maps to a specific technical remedy. For the synthetic training gap: collecting 50–100 real handwritten forms and including them in Phase 2 fine-tuning would introduce genuine stroke variability that font rendering cannot provide. For matra disambiguation: replacing greedy CTC decoding with beam search guided by a Nepali character-level language model would produce globally optimal character sequences biased toward valid Devanagari combinations, specifically improving recognition of rare matra combinations (Graves et al., 2006). For pixel-level matra visibility: morphological preprocessing — contrast enhancement and stroke normalisation — before CRNN inference would amplify the one-pixel height differences that currently fall below the model's discriminable threshold. For first name correction: a frequency-weighted vocabulary lookup would allow common names such as राम to be preferred over equally close but rarer candidates, enabling reliable first name correction without the over-correction observed during live testing.

8. Deployment and Practical Considerations

8.1 Hardware and Software Requirements

The training was done on one Nvidia T4 GPU (15 GiB VRAM) using the free GPU tier provided by Kaggle. However, because the PyTorch model does not require GPUs to run the model at inference time, it is technically possible to deploy to the GPU-less standard government office PC or mid-range laptop without needing to buy a GPU. It features all open-source and pip-installable components: PyTorch, torchvision, OpenCV, PyMuPDF, and Gradio, without any proprietary licenses, cloud subscriptions or dedicated hardware.

8.2 Connectivity Requirements

Once the model checkpoint and vocabulary file are found locally, the system runs entirely off of the internet. No need to have Internet connection at run-time. This is a conscious design decision: Many offices of the district level government in Nepal do not have reliable or adequate Internet connection (Rajkarnikar, 2025) and if the system had to rely on cloud APIs, it would be impossible to use it where it is needed the most. The Gradio web interface ([gui.py](#)) is not run on a server, but is served on the local machine (localhost, <http://127.0.0.1:7860>) from which it can be accessed by using any browser running on the local machine.

8.3 User Interface and Literacy Considerations

The Gradio web interface enables you to drag and drop an image and get a table of field predictions out. For deployment in Nepali government offices, interface labels and instructions should be in Devanagari script and not in English as the end users are government staff and are mainly Nepali speaking. The output of all models is in Devanagari (Devnagari script is used by all models) and the operator for reviewing and correcting predictions should be devnagari literate. This is a reasonable assumption to make for the intended users, but it would be best to put this in any deployment documentation.

8.4 Cost Estimate

Training cost is zero: both phases were completed on Kaggle's free GPU tier. Re-training with additional real handwritten data would incur the same zero monetary cost, assuming continued access to free Kaggle GPU quota. Deployment cost is zero for the software itself — the system is self-hosted on a local PC with no cloud hosting fee. Practical ongoing costs are human: operator time for mandatory field review and correction, and annotation labour required to produce labelled training examples from new handwritten data.

8.5 Data Privacy and Equity Considerations

Personal information (full names, nationalities, and the district in which the person resides) is included in the forms that are submitted via this method. Images of form scans should not be logged, sent, or stored following the processing session without a specific institutional data governance policy; all deployments in a government setting must comply with Nepal's applicable data protection laws. The vocabulary is another issue related to vocabulary equity. Popular Nepali names and the 77 administrative districts make up the JSON corpus. Error rates may be higher for names of ethnic groups that are underrepresented in the vocabulary, names of minority linguistic groups, or names of foreign origin. To find and fix disparities, large-scale deployments should examine and report on correction rates broken down by name origin.

9. Conclusion and Future Work

9.1 Summary

```
=====
FINAL RESULTS SUMMARY
=====

Phase 1 - Hindi Handwriting Test
Word Accuracy : 0.6731 (67.31%)
CER           : 0.0899 (8.99%)

Phase 2 - Nepali Font Fine-tune Test (before NLP)
Word Accuracy : 0.6992 (69.92%)
CER           : 0.0671 (6.71%)
Δ vs P1 base  : +22.76% word accuracy

Phase 3 - After NLP Postprocessing
Word Accuracy : 0.9187 (91.87%)
CER           : 0.0349 (3.49%)
Δ vs P2       : +21.95% word accuracy
=====

Checkpoints saved in:
/kaggle/working/checkpoints/phase1
/kaggle/working/checkpoints/phase2
```

Figure 13: Final Result Summery of both Phases

Lack of guarantee that people in rural areas can access services that rely on handwritten records from the government is known as the "documentation/equity issue." This project developed an End-to-End OCR pipeline which will solve the problem by using a fixed template Nepali membership form. Based on the seven-step pipeline (from form alignment using ORB to character recognition using CRNN and post-processing using NLP), a handwritten Devanagari OCR system can be built and tested using public data sets, without any proprietary tools and only using free compute resources (low resource system). The key technical result is that the two-stage transfer learning framework with a pretrained Hindi model yields a marked improvement in the word accuracy on the Nepali validation set. This model has a high accuracy and is a great value of the chain with 47. It has been shown to be 26% accurate in Nepali and its improvements were controlled and attributable (Yosinski et al., 2014). Two real handwritten test forms resulted in a clean zero regression correction, better results in three fields and zero additional error in the NLP post-processor. These are helpful, but limited: the two forms used in the realistic evaluation are common, have different sets of training examples, and approximately half of the words—the 52—remain incorrect at inferences. The 23 percent result is obtained by using data that are artificially created in a font. This system is a support system for the first-pass digitization which is used in combination with human verification, and is not intended to replace human transcription.

9.2 Future Work

The easiest methods to enhance the system are listed in the three points below. The stroke variability (slant, ink spread, and connected letterforms) will be considered a direct solution to the primary domain gap if 50–100 handwritten Nepali forms are gathered and incorporated

into Phase 2 fine tuning. This is because rendering the synthetic hands alone is not sufficient to address the primary domain gap. Second, rather than employing greedy CTC decoding, a language model utilizing Nepali characters would produce globally optimal sequences, which would also aid in identifying unusual matra combinations (Graves et al. (2006)) Third, morphological preprocessing of the stroke before CRNN inference would increase the height difference between short and long vowel signs, which are indistinguishable at the current

resolution of 320x128 because they are only one pixel high. Along with these initial enhancements, the system could be made practically ready for a nationwide rollout by adding minority community surnames and district-specific name distribution to the vocabulary set and expanding from a fixed template form to a free form document understanding.

10. References

Additional resources for the Project are at:

<https://drive.google.com/drive/folders/1PKDdDji6OBmDyi3dMoagNzaCXLougXli?usp=sharing>

- Back, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character region awareness for text detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 9365–9374. <https://doi.org/10.1109/CVPR.2019.00959>
- Department of National ID and Civil Registration (DONIDCR). (2023). *Integrated social registry: Expression of interest*. Government of Nepal, Ministry of Home Affairs. https://donidcr.gov.np/Files/ISR_%20EOI_Finalbef1a12d-0623-4ef1-9899-3a6b65e3a6d6.pdf
- DONIDCR. (2025). *National ID registration in Nepal*. <https://citizenportal.donidcr.gov.np/en>
- Deshmukh, A., Ingle, M., & Bharkad, S. (2020). Isolated handwritten Devanagari character recognition using VGG16. *International Journal of Advanced Science and Technology*, 29(5), 7965–7977.
- Dhakal, A. (2022, April 4). Saving Nepal's archives from oblivion. *Nepali Times*. <https://nepalitimes.com/saving-nepal-s-archives-from-oblivion>
- Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of the 23rd International Conference on Machine Learning*, 369–376. <https://doi.org/10.1145/1143844.1143891>
- International Organization for Migration (IOM). (2016). *Barriers to women's land and property access and ownership in Nepal*. IOM. <https://www.iom.int/resources/barriers-womens-land-and-property-access-and-ownership-nepal>
- kritakhere. (n.d.). *Nepali font generated handwritten names* [Data set]. Kaggle. <https://www.kaggle.com/datasets/kritakhere/nepali-font-generated-handwritten-names>
- Masrat, S., Wani, M. A., & Bhat, M. A. (2022). Devanagari handwritten character recognition using VGG-16 convolutional neural network. *Multimedia Tools and Applications*, 81(22), 31625–31640. <https://doi.org/10.1007/s11042-022-12685-5>
- Myrepublica. (2021, April 15). Efforts at digitizing government services in current federal set-up. *Nagarik Network*. <https://myrepublica.nagariknetwork.com/news/government-efforts-at-digitizing-its-services-are-paying-off/>
- Rajkarnikar, R. (2025). Digitalisation of public services in Nepal: An assessment of the implementation status. *Nepalese Journal of Economic Studies*, 4(1), 99–121. <https://doi.org/10.3126/njes2.v4i1.82964>
- Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). ORB: An efficient alternative to SIFT or SURF. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2564–2571. <https://doi.org/10.1109/ICCV.2011.6126544>
- Sabarinathan. (n.d.). *Handwritten Hindi word recognition dataset* [Data set]. Kaggle. <https://www.kaggle.com/datasets/sabarinathan/handwritten-hindi-word-recognition>
- Shi, B., Bai, X., & Yao, C. (2016). An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), 2298–2304. <https://doi.org/10.1109/TPAMI.2016.2646371>

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition.
Proceedings of the 3rd International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1409.1556>

Smith, R. (2007). An overview of the Tesseract OCR engine.
Proceedings of the 9th International Conference on Document Analysis and Recognition, 629–633.
<https://doi.org/10.1109/ICDAR.2007.4376991>

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?
Advances in Neural Information Processing Systems, 27.
<https://proceedings.neurips.cc/paper/2014/hash/375c71349b295fbe2dcdca9206851369-Abstract.html>

Appendix A — Contribution Log

Note: The contribution log must be completed honestly. The assessor may cross-check against meeting minutes in Appendix B.

Member	T1 Proposal	T2 Analysis	T3 Code	T4 Report	T5 Presentation
Kritak Aryal (Lead)	Drafted proposal & scope	Documented comparison	Built full pipeline (ORB, CRNN, demo)	Implementation & Results	Solution & pipeline walkthrough
Rashmi Gauli	Reviewed existing tools	Led critical analysis	Compiled Nepali vocabulary	Existing-solutions section	Existing solutions & comparison
Junisha Shakya	Proposal sections	Analysis & referencing	Testing support	Structure & referencing	Proposed solution walkthrough
Noel Maharjan	Proposal sections	Limitation logging	Font verification & testing	Deployment & self-critique	Results & reflection

Appendix B — Meeting Minutes

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 1 · Topic Finalization
Date / Time	Wednesday, 2 July 2025 · 12:00 PM
Mode / Venue	On-Premise · IIMS College, Naxal, Kathmandu
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Reviewed ITS69204 assignment brief; identified NLP + computer vision integration as the core requirement. 2. Brainstormed project domains: handwritten text recognition, sentiment analysis, image captioning. 3. Discussed Nepali language digitisation as a high-impact, real-world problem. 4. Evaluated each domain against available datasets, compute resources, and group expertise. 5. Reached consensus on Nepali handwritten OCR for fixed-template government forms as the project topic.
DECISIONS & OUTCOMES	
Agreed	✓ Project topic confirmed: End-to-end Nepali handwritten OCR pipeline. ✓ Scope defined as Devanagari character recognition from scanned form images. ✓ Next step: literature search to identify three existing OCR solutions for comparison.
ACTION ITEMS	
Tasks	☐ [All] Review related OCR papers before next meeting. ☐ [Kritak] Draft initial project scope statement and share with group.

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 2 · Solution Search & Existing System Review
Date / Time	Saturday, 5 July 2025 · 7:00 PM
Mode / Venue	Online (Video Call) · Remote
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Each member presented findings from independent literature and tool searches. 2. Tesseract OCR: open-source, supports Devanagari, but weak on handwriting and lacks form-field extraction. 3. VGG16 as classifier: strong feature extractor but requires character-level segmentation as a pre-processing step. 4. EasyOCR: easy deployment, Devanagari support, but optimised for printed text; no template-based field extraction. 5. Compared all three on: handwriting robustness, Devanagari support, segmentation dependency, structured output. 6. Identified key gap: none of the three tools provide template-based structured field extraction for handwritten Devanagari.
DECISIONS & OUTCOMES	
Agreed	✓ Three comparison solutions selected: Tesseract OCR, VGG16-based classifier, EasyOCR. ✓ Gap confirmed: no existing tool handles end-to-end structured form-field extraction with handwritten Devanagari. ✓ Custom pipeline required to address the identified gap.
ACTION ITEMS	
Tasks	☐ [Kritak] Document the three-solution comparison for the Literature Review section. ☐ [All] Research CRNN architectures and CTC decoding for the next session.

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 3 · Architecture Planning & Scope Narrowing
Date / Time	July 2025 (exact date TBC) · 12:00 PM
Mode / Venue	On-Premise · IIMS College, Naxal, Kathmandu
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Discussed overall pipeline: form input → alignment → field crop → recognition → post-processing → JSON output. 2. Evaluated recognition architectures: pure CNN classifier, CRNN + CTC, transformer-based TrOCR. 3. Reviewed tools and frameworks: PyTorch, torchvision, OpenCV, PyMuPDF, Gradio. 4. Debated scope: free-form document recognition vs. fixed-template form with predefined field regions. 5. Agreed free-form approach introduces unresolvable complexity given timeline and data constraints. 6. Nine target fields listed: first name, last name, place of birth, father's first/last name, mother's first/last name, nationality, district.
DECISIONS & OUTCOMES	
Agreed	✓ Architecture confirmed: ORB alignment → coordinate-based crop → CRNN (VGG16 + BiLSTM + CTC) → NLP post-processing. ✓ Scope narrowed: single fixed-template form with nine predefined field regions. ✓ Stack confirmed: PyTorch, OpenCV, PyMuPDF, Levenshtein, Gradio. Free-form recognition explicitly out of scope.
ACTION ITEMS	
Tasks	☐ [Kritak] Implement ORB alignment and field cropping modules. ☐ [Kritak] Set up CRNN model skeleton in PyTorch. ☐ [All] Confirm nine field bounding-box coordinates on the reference form template.

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 4 · Dataset Search & Limitation Logging
Date / Time	July 2025 (exact date TBC) · 8:00 PM
Mode / Venue	Online (Video Call) · Remote
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Conducted structured search across Kaggle, UCI ML Repository, Papers With Code, and Google Dataset Search. 2. Finding: no dedicated handwritten Nepali word recognition dataset of sufficient size or quality exists publicly. 3. Existing Devanagari datasets are character-level (isolated glyphs) or Hindi-word-level — not Nepali. 4. Discussed implications: training from scratch on Nepali is infeasible; transfer learning from a related script is required. 5. Explored three options: (a) collect real handwritten forms, (b) Hindi dataset + transfer learning, (c) synthetic Nepali data generation. 6. Formally logged the dataset gap as a documented project limitation.
DECISIONS & OUTCOMES	
Agreed	✓ Confirmed: no suitable handwritten Nepali word dataset exists publicly. ✓ Limitation documented: synthetic or transfer-based training is the only feasible path. ✓ Decision on specific strategy deferred to Meeting 5 pending further investigation.
ACTION ITEMS	
Tasks	☐ [Kritak] Investigate HindiSeg and comparable Hindi handwriting datasets on Kaggle. ☐ [Rashmi] Compile a list of common Nepali first names, surnames, and district names. ☐ [All] Research font-based synthetic data generation techniques for Devanagari.

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 5 · Dataset Finalization & Synthetic Generation Decision
Date / Time	July 2025 (exact date TBC) · 7:00 PM
Mode / Venue	Online (Video Call) · Remote
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Kritak presented HindiSeg dataset: ~92,000 real handwritten Hindi word images, clean labels, Devanagari script (Kaggle). 2. Agreed to use HindiSeg for Phase 1 pre-training, exploiting Hindi–Nepali script identity. 3. Rashmi and Noel presented collected Nepali vocabulary: first names, surnames, all 77 district names. 4. Discussed synthetic generation: render Nepali words using 18 Devanagari font variants at multiple sizes. 5. Agreed augmentation pipeline: rotation $\pm 5^\circ$, Gaussian blur, brightness/contrast jitter, salt-and-pepper noise, perspective distortion. 6. Estimated Phase 2 dataset size: ~24,000 word images. Train/validation split: 85%/15%, stratified across vocabulary.
DECISIONS & OUTCOMES	
Agreed	✓ Phase 1 dataset confirmed: HindiSeg (~92,000 real handwritten Hindi word images). ✓ Phase 2 dataset confirmed: synthetic Nepali words (~24,000 images, 18 font variants, full augmentation pipeline). ✓ Vocabulary finalised: Nepali first names, surnames, and 77 district names.
ACTION ITEMS	
Tasks	☐ [Kritak] Implement synthetic dataset generation script with augmentation pipeline. ☐ [Kritak] Set up Kaggle notebook for Phase 1 training on HindiSeg. ☐ [Rashmi] Clean and finalise the compiled Nepali vocabulary list. ☐ [Noel] Verify 18 font variants render all required Devanagari glyphs correctly.

MEETING MINUTES ITS69204 – Nepali Handwritten OCR

MEETING DETAILS	
Meeting	Meeting 6 · Training Evaluation & Outcome Discussion
Date / Time	July 2025 (exact date TBC) · 7:30 PM
Mode / Venue	Online (Video Call) · Remote
Attendees	Kritak (Leader), Rashmi, Junisha, Noel
DISCUSSION SUMMARY	
Points	1. Phase 1 results: CRNN pre-trained on HindiSeg achieved convergent CTC loss; phase1 checkpoint saved. 2. Phase 2 results: best checkpoint (crnn_nepali_best_phase2b.pth) achieved 52.23% word accuracy on synthetic Nepali val set. 3. Hindi-only baseline on Nepali val: ~47.2% — confirming +4.97 pp gain from fine-tuning. 4. Primary error modes identified: matra confusion (ि vs ी), chandrabinu errors, conjunct substitutions. 5. Live demo (2 real forms): Form 1 → 78% field accuracy after NLP post-processing; Form 2 → 89%. 6. NLP ablation: 3 fields corrected (all श्रेष्ठ misrecognitions); 0 new errors introduced across both forms. 7. Consistent failure: जन्मस्थान — काठमाडौं not recognised due to chandrabinu + cursive stroke joining. 8. Deployment consensus: viable as first-pass digitisation aid with mandatory human review; not autonomous.
DECISIONS & OUTCOMES	
Agreed	✓ Primary model confirmed: crnn_nepali_best_phase2b.pth (52.23% synthetic validation accuracy). ✓ Transfer learning confirmed effective: +4.97 pp over Hindi-only baseline. ✓ NLP post-processor confirmed zero-regression: 3 corrections, 0 new errors on real test forms. ✓ जन्मस्थान failure logged as a known limitation for future work.
ACTION ITEMS	
Tasks	☐ [Kritak] Write Implementation and Results sections of the final report. ☐ [All] Complete assigned report sections before the submission deadline. ☐ [Kritak] Finalise Gradio GUI and verify live demo reproducibility.

Appendix C- Project Proposal

PROJECT PROPOSAL¶

Handwritten OCR for Nepali Government Forms: A CRNN Pipeline with NLP Post-Processing¶

¶

Group Members: -Kritak (Leader), Rashmi, Junisha, Noel¶

Module: -ITS69204 - Computer Vision and Natural Language Processing¶

Institution: -IIMS College, Naxal, Kathmandu - Taylor's University Malaysia¶

Date: -2 July 2025¶

1. Problem Statement¶

Nepal holds a large volume of handwritten government records — citizenship documents, land registries, and district administrative forms — almost none of which exist in digital form. Manual transcription is slow, expensive, and hard to scale. The Nepali language is written in Devanagari script, where vowel sounds appear as diacritics attached to consonant characters and consonant clusters form conjunct glyphs that differ substantially from their individual parts. This makes handwritten Nepali considerably harder to recognise than handwritten Latin script, and most existing OCR tools are not designed for it. We propose to build a recognition system that takes a scanned government form as input and automatically extracts the handwritten field values, reducing the transcription burden on staff.¶

2. Existing Solutions We Are Exploring¶

Before settling on our own approach, we are reviewing existing tools and techniques to understand what is already available and where the gaps are. Tesseract is an open-source OCR engine with Devanagari support and wide use in printed document digitisation; we are exploring whether it can handle handwritten input, though we expect its performance on handwriting to be limited. EasyOCR is a more recent deep learning-based OCR library that also supports Devanagari and is easier to deploy; we plan to test it on handwritten Nepali samples to see how it performs out of the box. We are also exploring CNN-based recognition pipelines, and the idea of combining a convolutional feature extractor with a recurrent sequence model and Connectionist Temporal Classification (CTC) decoding stands out to us as particularly promising — it avoids the need to segment individual characters before recognition, which we understand to be one of the harder steps in the problem.¶

3. Our Proposed Approach¶

Based on what we know so far, our plan is to build a pipeline that identifies the handwritten field regions of a scanned form and passes each region through a recognition model to produce a text output. We intend to explore a CNN with sequence modelling and CTC decoding as the core recogniser, and compare its performance against the off-the-shelf tools above. We are still working through the dataset question — we are looking for a publicly available handwritten Nepali or Devanagari word dataset, and if Nepali-specific data turns out to be scarce, we may consider whether a Hindi handwriting dataset could serve as a starting point given that both languages share the same script. We also plan to incorporate some form of post-recognition correction, where the raw output is checked against a list of known Nepali words or names to catch likely errors — this is where we see NLP contributing most naturally alongside the computer vision pipeline.¶

4. Team and Responsibilities¶

We have divided initial responsibilities as follows, though we expect roles to shift as we learn more:¶

Kritak (Leader): - Overall pipeline design, model implementation, and technical decisions.¶

Rashmi: - Existing solution review and dataset research.¶

Junisha: - Documentation, limitation logging, and report writing.¶

Noel: - Tool evaluation, testing, and implementation support.¶

5. Expected Outcomes¶

By the end of this project we expect to have a working prototype that accepts a scanned handwritten Nepali form and returns the extracted field values as structured text output. We also expect to have a clear comparison of our approach against the existing tools we are evaluating, and an honest account of where our system succeeds and where it does not, with specific proposals for what could be improved with more time or data.¶

Appendix D — Peer Evaluation Summary

Member	Own Rating	Average Peer Rating	Notable Feedback Themes
Kritak Aryal	9/10	8/10	Led architecture & implementation; reliable; drove technical decisions
Rashmi Gauli	9/10	8/10	Strong on research & analysis; owned vocabulary and comparison work
Junisha Shakya	9/10	8/10	Documentation & referencing quality; kept report structure consistent
Noel Maharjan	9/10	8/10	Reliable on testing/verification; contributed to deployment discussion

Appendix D — AI Tool Usage Declaration

Item	Details
Tools used	Claude (Coding Support and Report help) ChatGPT (Pipeline Architecture design to Image)
Sections used for	Structural drafting of Sections 1–10; Used Claude as aid to code autocomplete in implementation scripts. Generation of “Nepali names in handwritten font, dataset”
How it was used	Structured Outline, Work polishing (grammar corrections, vocabulary enhancement, and sentence restructuring) All outputs were reviewed and edited by our team All metric values verified independently against training logs
What the team learned (While Using Ai)	How to frame critical analysis arguments How to structure APA 7 methodology sections How to present ablation results clearly
What AI was NOT used for	All metric values, architecture decisions, experiment design, actual training-and-testing runs, live demo execution, or real-form evaluation

Declaration: AI assistance was limited to language and presentation. All experimental design, data collection, model training, evaluation, and results are entirely the team's own.